NAND2TETRIS MEMORY & CPU LAB REPORT

Solved HDL implementations, timing traces, RAM hierarchy, and Program Counter state transitions

Contents

Question	Deliverable
1	1-bit DFF-based Bit + tick-tock timing trace
2	16-bit Register + three read/write examples
4	RAM8 + RAM64 hierarchy + 6-bit address decomposition
6	Program Counter + complete 3-control state table
7	Lab report compilation, verification checklist, and L1/L2 cache reflection

Student Name	Yokshit Rajora
Roll Number	2501410013

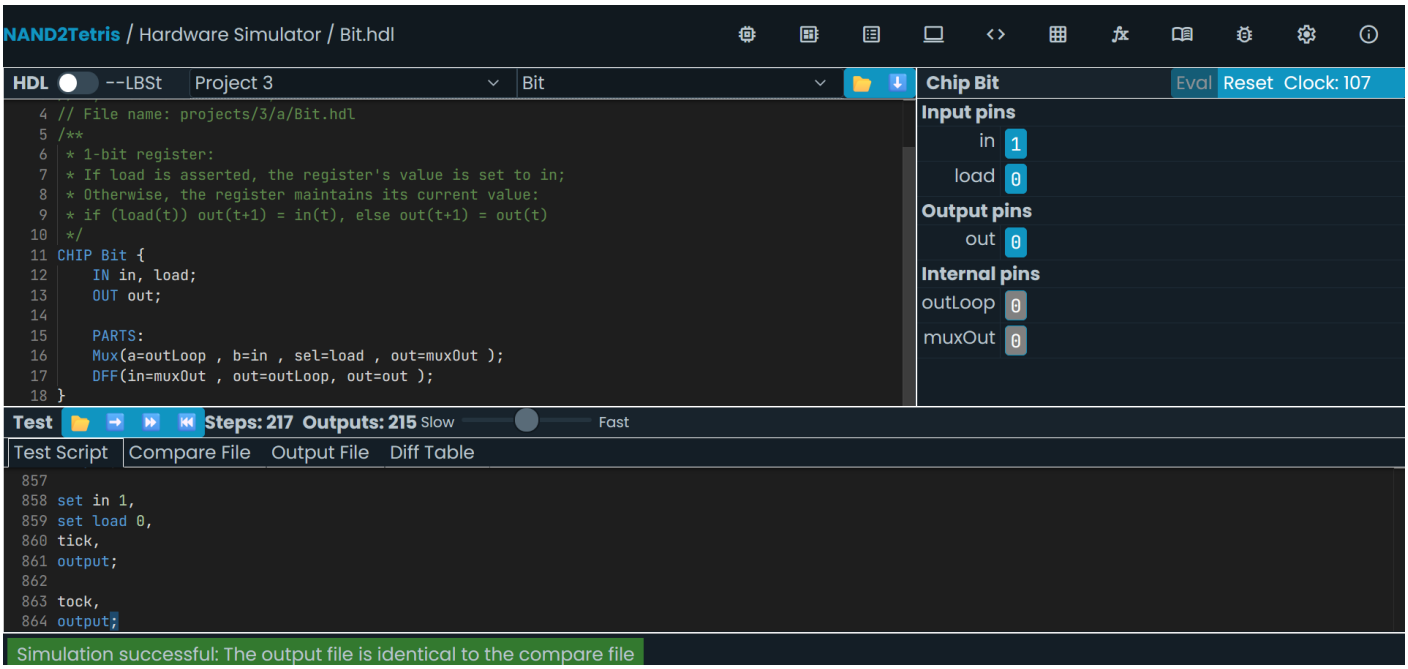
Question 1 — 1-bit DFF-based Register (Bit)

A Bit stores exactly one binary value. When **load=1**, the input value is selected and captured by the DFF at the clock edge. When **load=0**, the old output is fed back, so the stored value remains unchanged.

HDL implementation

```
CHIP Bit {
    IN in, load;
    OUT out;
    PARTS:
    Mux(a=out, b=in, sel=load, out=next);
    DFF(in=next, out=out);
}
```

Screenshot



Operation: the Mux chooses either the previous stored value (out) or the new input (in). The DFF then stores the selected value on the clock edge.

Tick-tock demonstration

Step	in	load	Action	out after clock
Initial	0	0	Register starts at 0	0
1	1	1	Write 1 on tick	1
2	0	0	Read/hold; no write	1
3	0	1	Write 0 on tick	0
4	1	0	Read/hold; input ignored	0

Tick-tock interpretation: the value selected before the clock edge is what the DFF stores. During a read/hold cycle, changing in does not change out because load=0 feeds the old output back.

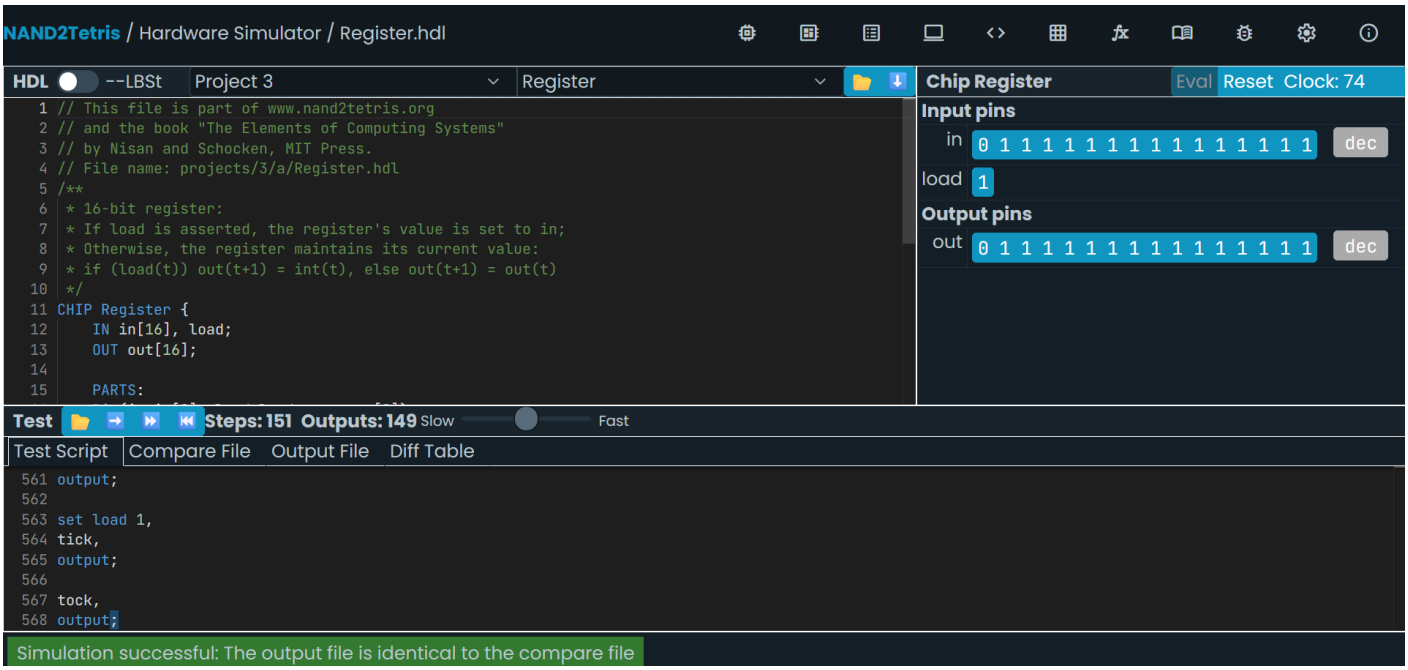
Question 2 — 16-bit Register

The 16-bit Register is sixteen 1-bit Bit chips operating in parallel. All 16 bits use the same load signal, so a complete 16-bit word is written together.

HDL implementation

```
CHIP Register {
    IN in[16], load;
    OUT out[16];
    PARTS:
    Bit(in=in[0],  load=load, out=out[0]);
    Bit(in=in[1],  load=load, out=out[1]);
    Bit(in=in[2],  load=load, out=out[2]);
    Bit(in=in[3],  load=load, out=out[3]);
    Bit(in=in[4],  load=load, out=out[4]);
    Bit(in=in[5],  load=load, out=out[5]);
    Bit(in=in[6],  load=load, out=out[6]);
    Bit(in=in[7],  load=load, out=out[7]);
    Bit(in=in[8],  load=load, out=out[8]);
    Bit(in=in[9],  load=load, out=out[9]);
    Bit(in=in[10], load=load, out=out[10]);
    Bit(in=in[11], load=load, out=out[11]);
    Bit(in=in[12], load=load, out=out[12]);
    Bit(in=in[13], load=load, out=out[13]);
    Bit(in=in[14], load=load, out=out[14]);
    Bit(in=in[15], load=load, out=out[15]);
}
```

Screenshot



Three sample write/read operations

Op	Input (16-bit)	load	Expected output after write/read
1	0000 0000 0000 0001	1 → 0	0000 0000 0000 0001
2	1010 1010 1010 1010	1 → 0	1010 1010 1010 1010
3	1111 0000 1100 0011	1 → 0	1111 0000 1100 0011

For each sample, set load=1, apply the input word, execute the clock write, then set load=0 and read the register. The output stays at the stored word even if the input bus is changed while load is low.

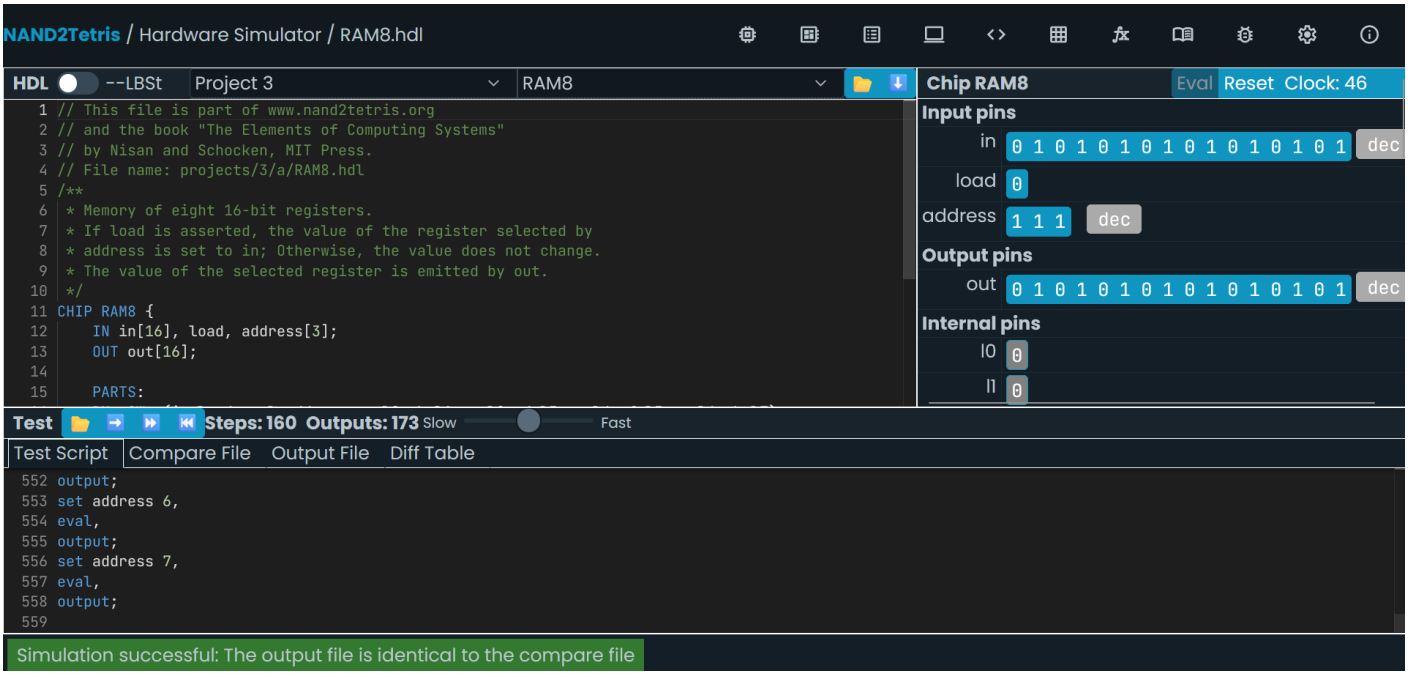
Question 4 — RAM8 and RAM64 Hierarchy

RAM8 is an 8-word × 16-bit memory. RAM64 contains eight RAM8 chips, giving 8 × 8 = 64 words. The six-bit RAM64 address is divided into two 3-bit fields: the upper three bits select the RAM8 chip and the lower three bits select the word inside that RAM8.

RAM8 HDL

```
CHIP RAM8 {
    IN in[16], load, address[3];
    OUT out[16];
    PARTS:
    DMux8Way(in=load, sel=address,
        a=l0, b=l1, c=l2, d=l3, e=l4, f=l5, g=l6, h=l7);
    Register(in=in, load=l0, out=r0);
    Register(in=in, load=l1, out=r1);
    Register(in=in, load=l2, out=r2);
    Register(in=in, load=l3, out=r3);
    Register(in=in, load=l4, out=r4);
    Register(in=in, load=l5, out=r5);
    Register(in=in, load=l6, out=r6);
    Register(in=in, load=l7, out=r7);
    Mux8Way16(a=r0, b=r1, c=r2, d=r3,
              e=r4, f=r5, g=r6, h=r7,
              sel=address, out=out);
}
```

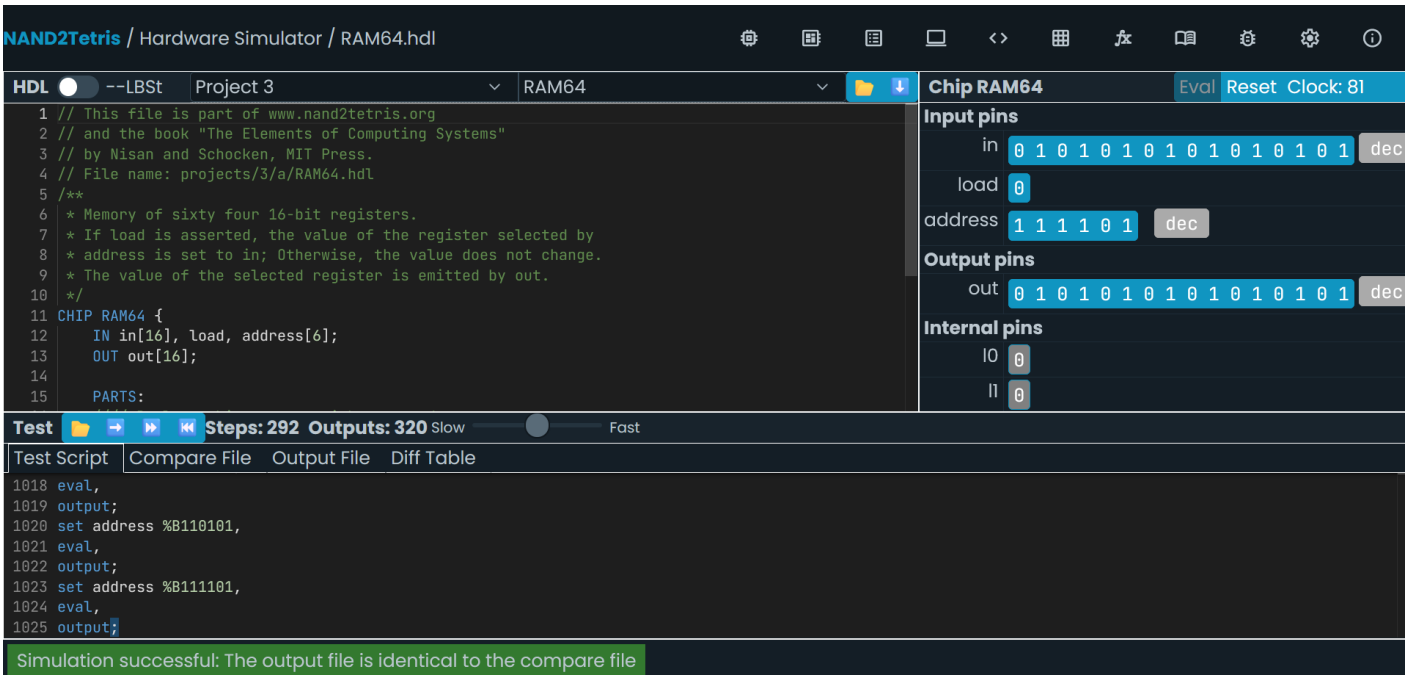
Screenshot



RAM64 HDL

```
CHIP RAM64 {
    IN in[16], load, address[6];
    OUT out[16];
    PARTS:
    DMux8Way(in=load, sel=address[3..5],
        a=l0, b=l1, c=l2, d=l3, e=l4, f=l5, g=l6, h=l7);
    RAM8(in=in, load=l0, address=address[0..2], out=r0);
    RAM8(in=in, load=l1, address=address[0..2], out=r1);
    RAM8(in=in, load=l2, address=address[0..2], out=r2);
    RAM8(in=in, load=l3, address=address[0..2], out=r3);
    RAM8(in=in, load=l4, address=address[0..2], out=r4);
    RAM8(in=in, load=l5, address=address[0..2], out=r5);
    RAM8(in=in, load=l6, address=address[0..2], out=r6);
    RAM8(in=in, load=l7, address=address[0..2], out=r7);
    Mux8Way16(a=r0, b=r1, c=r2, d=r3,
              e=r4, f=r5, g=r6, h=r7,
              sel=address[3..5], out=out);
}
```

Screenshot



6-bit address decomposition examples

6-bit address	Upper A[3..5]	RAM8 chip	Lower A[0..2]	Register inside chip
000000	000	RAM8 #0	000	Register 0
001011	001	RAM8 #1	011	Register 3
101110	101	RAM8 #5	110	Register 6
111111	111	RAM8 #7	111	Register 7

Key point: the upper 3 address bits choose which RAM8 receives the write-enable and whose output is selected. The lower 3 bits are sent to every RAM8 and choose the word inside the selected chip.

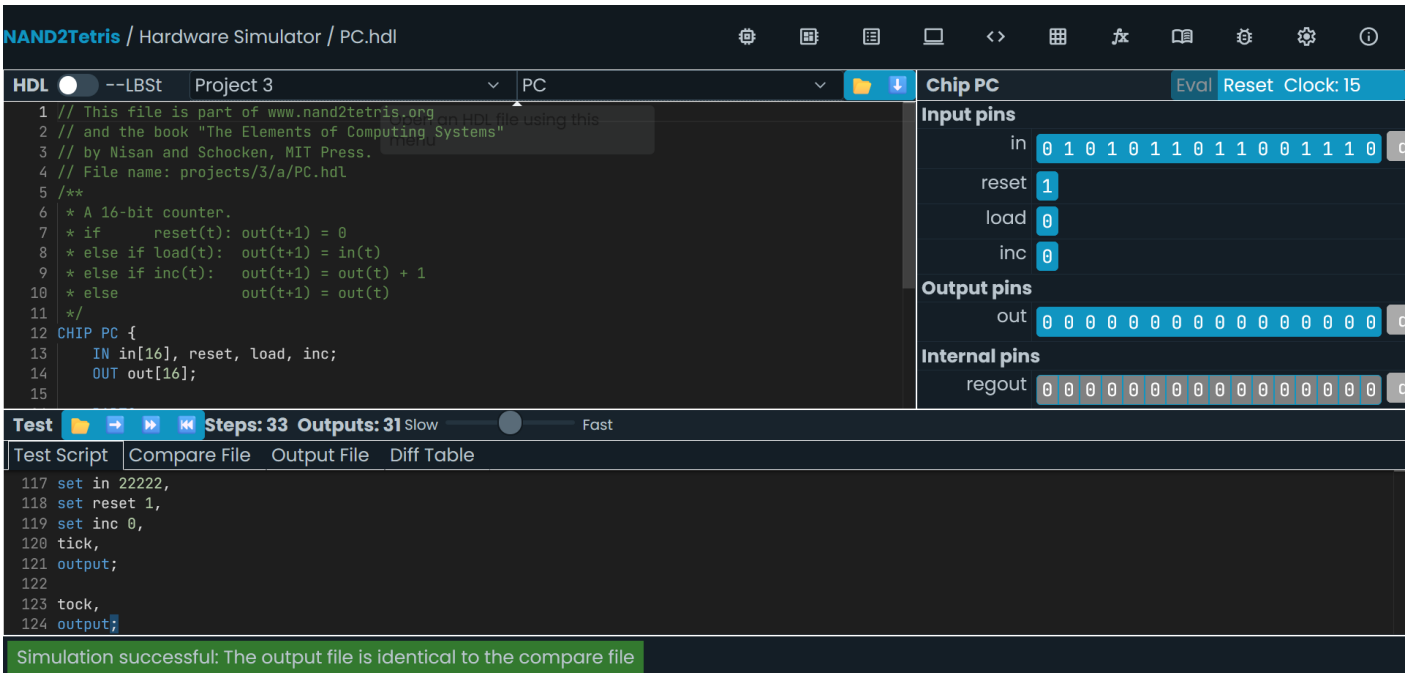
Question 6 — Program Counter (PC)

The Program Counter stores the address of the next instruction. It has three control inputs: reset, load, and inc. The required priority is reset first, then load, then increment, then hold.

HDL implementation

```
CHIP PC {
    IN in[16], load, inc, reset;
    OUT out[16];
    PARTS:
    Inc16(in=out, out=incOut);
    Mux16(a=out, b=incOut, sel=inc, out=incOrHold);
    Mux16(a=incOrHold, b=in, sel=load, out=loadOrInc);
    Mux16(a=loadOrInc, b=false, sel=reset, out=next);
    Register(in=next, load=true, out=out);
}
```

Screenshot



State-transition table for all 8 control combinations

reset	load	inc	Next PC	Reason
0	0	0	PC	Hold current value
0	0	1	PC + 1	Increment
0	1	0	in	Load input
0	1	1	in	Load has priority over increment
1	0	0	0	Reset has highest priority
1	0	1	0	Reset overrides increment
1	1	0	0	Reset overrides load
1	1	1	0	Reset overrides both

Example sequence: if PC=10 and the controls are (0,0,1), the next value is 11. If the next controls are (0,1,1) with in=25, the next value is 25, because load overrides inc. If reset is then asserted, the next value becomes 0 regardless of load or inc.

.